

CRUX: HOW TO CREATE AND CONTROL THREADS

What interfaces should the OS present for thread creation and control? How should these interfaces be designed to enable ease of use as well as utility?

Thread Creation:

```
#include <pthread.h>
int
pthread_create(pthread_t *thread,
               const pthread_attr_t *attr,
               void *(*start_routine)(void*),
               void *arg);
```

1. **thread**: pointer to a structure `pthread_t`; we'll use this to interact with this thread.
2. **attr**: used to specify any attributes this thread might have such as stack size, scheduling priority.
3. Which function should this thread start running in? In C, this is pointer functions.
4. **arg**: is the argument passed to the thread where the thread begins execution.

`void` pointer allows us to pass/return any type of argument.

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  typedef struct {
5      int a;
6      int b;
7  } myarg_t;
8
9  void *mythread(void *arg) {
10     myarg_t *args = (myarg_t *) arg;
11     printf("%d %d\n", args->a, args->b);
```

```

11     printf("a=%d\n", args->a, args->b);
12     return NULL;
13 }
14
15 int main(int argc, char *argv[]) {
16     pthread_t p;
17     myarg_t args = { 10, 20 };
18
19     int rc = pthread_create(&p, NULL, mythread, &args);
20     ...
21 }

```

Figure 27.1: Creating a Thread

Thread Completion

```
int pthread_join(pthread_t thread, void **value_ptr);
```

The second argument is a pointer to the return value.

As the routine can return anything, it is defined to return a pointer to void; because the pthread_join() routine changes the value of the passed argument, you need to pass a pointer to that value & not just value itself.

```

1  typedef struct { int a; int b; } myarg_t;
2  typedef struct { int x; int y; } myret_t;
3
4  void *mythread(void *arg) {
5      myret_t *rvals = Malloc(sizeof(myret_t));
6      rvals->x = 1;
7      rvals->y = 2;
8      return (void *) rvals;
9  }
10
11 int main(int argc, char *argv[]) {
12     pthread_t p;
13     myret_t *rvals;
14     myarg_t args = { 10, 20 };
15     Pthread_create(&p, NULL, mythread, &args);
16     Pthread_join(p, (void **) &rvals);
17     printf("returned %d %d\n", rvals->x, rvals->y);
18     free(rvals);
19     return 0;
20 }

```

Figure 27.2: Waiting for Thread Completion

Importantly, never return a pointer which refers to something allocated on the thread's call stack.

```
1 void *mythread(void *arg) {  
2     myarg_t *args = (myarg_t *) arg;  
3     printf("%d %d\n", args->a, args->b);  
4     myret_t oops; // ALLOCATED ON STACK: BAD!  
5     oops.x = 1;  
6     oops.y = 2;  
7     return (void *) &oops;  
8 }
```

In this case, the variable `oops` is allocated on the stack of `mythread()`.

However, when it returns, the value is automatically deallocated, and thus passing back a pointer to a now deallocated variable.

Locks

Posix threads library provides functions for providing mutual exclusion to critical sections via locks.

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

If there is a region of code that is a critical section, it thus needs to be protected to ensure correct operation,

```
pthread_mutex_t lock;  
pthread_mutex_lock(&lock);  
x = x + 1; // or whatever your critical section is  
pthread_mutex_unlock(&lock);
```

In above code:

↳ if no thread holds the lock when `pthread_mutex_lock()` is called, the thread will acquire the lock & enter the critical section.

↳ If another thread does indeed hold the lock, the thread trying to grab the lock will not return the call unless the thread holding the lock releases via `unlock`.

↳ Many threads may be stuck inside the lock acquisition function; only thread with lock acquired should call `unlock`.

↳ There two ways to initialize locks:

- `pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;`

↳ This sets default values automatically

- `int rc = pthread_mutex_init(&lock, NULL);
assert(rc == 0); // always check success!`

↳ This initializes the lock at runtime

↳ The first argument is address of lock itself & the second argument is optional set of attributes.

↳ Lastly, when you are done with the lock call

`pthread_mutex_destroy()`

↳ Two other routines that interact with lock are:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_timedlock(pthread_mutex_t *mutex,  
                             struct timespec *abs_timeout);
```

These two calls are used in lock acquisition. The `trylock` version returns failure if the lock is already held; the `timedlock` version of acquiring a lock returns after a timeout or after acquiring the lock, whichever happens first. Thus, the `timedlock` with a timeout of zero degenerates to the `trylock` case. Both of these versions should generally be avoided; however, there are a few cases where avoiding getting stuck (perhaps indefinitely) in a lock acquisition routine can be useful, as we'll see in future chapters (e.g., when we study deadlock).

Read book, I am NOT writing more as other
chapters cover it.